6. Faster Python
- run as low-level code-

Department of Smart Mobility Engineering
Inwook, Shim
인하대학교
INHA UNIVERSITY

Previously

- Performance Counter
- Memory Fragmentation
- Vector computation
- In-place operation

Today

- Just in Time (JIT) and Ahead of Time (AOT)
- Cython
- pyximport
- Cython v.s. numpy
- OpenMP
- Numba

To make a faster code

- **Faster Code ?= Efficient Code**
- Doing the same amount of work with fewer instructions
  - Cython : C-based compiling approach
  - Numba : LLVM-based compiling approach (Low Level Virtual Machine)
  - PyPy : replacement virtual machine

    가상머신 기반의 저수준 언어 실행 환경을 목표로 시작된 프로젝트
  - GPU
  - CPython
  - Code down to machine code (Assembly 🤪)

Higher-level Algorithmic Optimization

- Mathematics level : reducing computation complexity itself.
- Implementation level :

  - remove redundancy (*e.g.*, re-calculation)

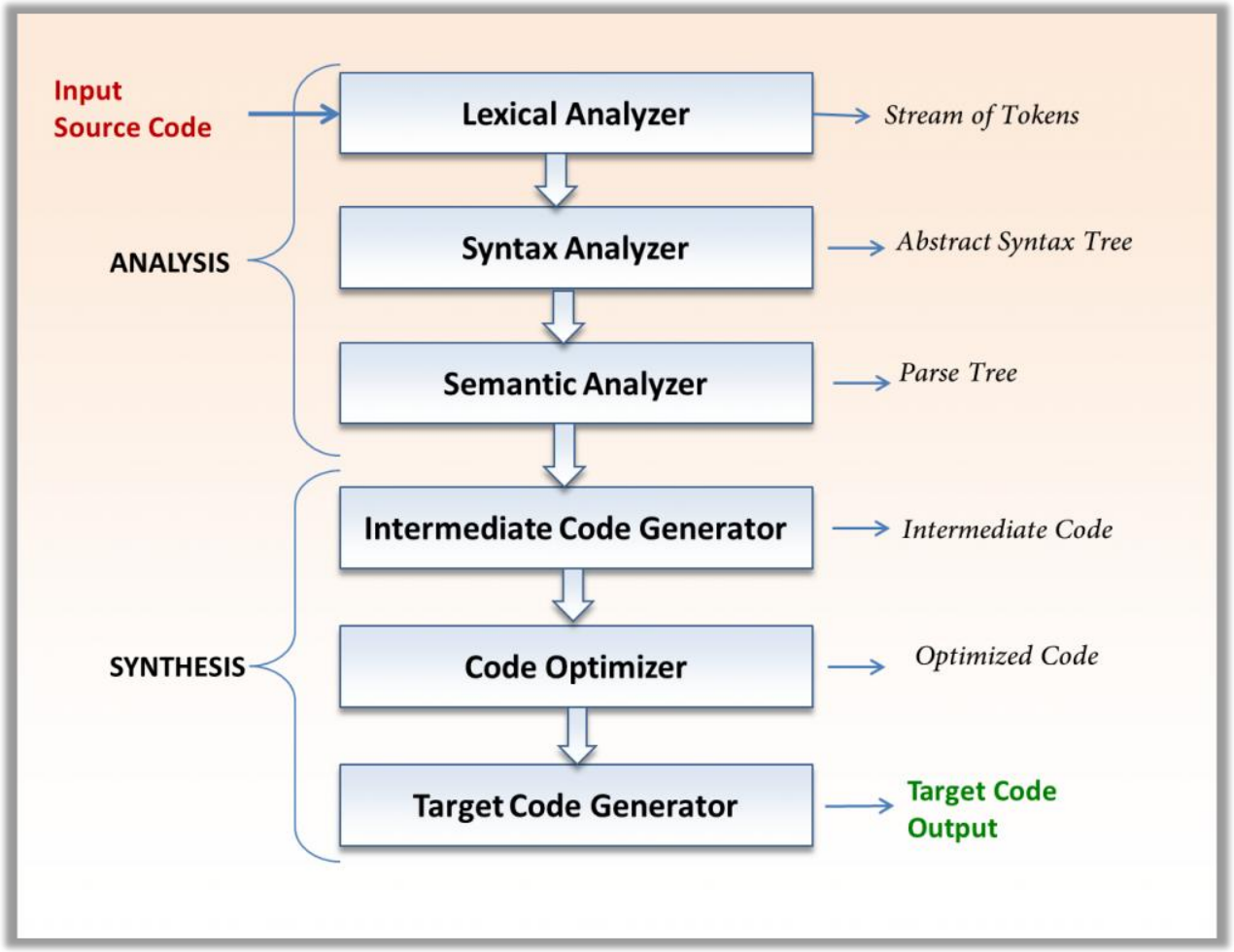
  - avoid unnecessary code

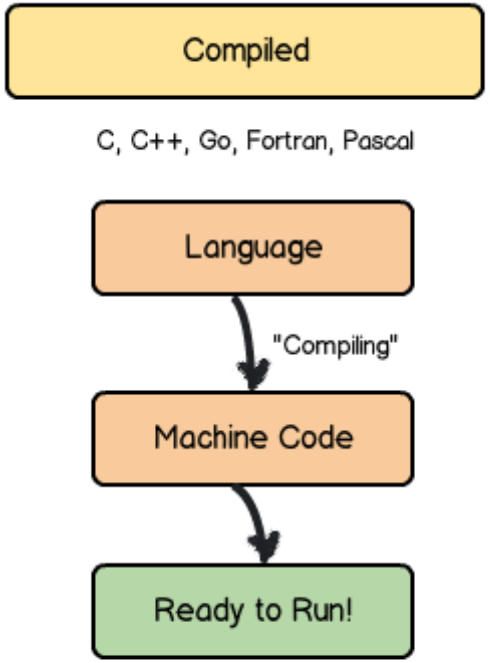
  - re-use cache memory

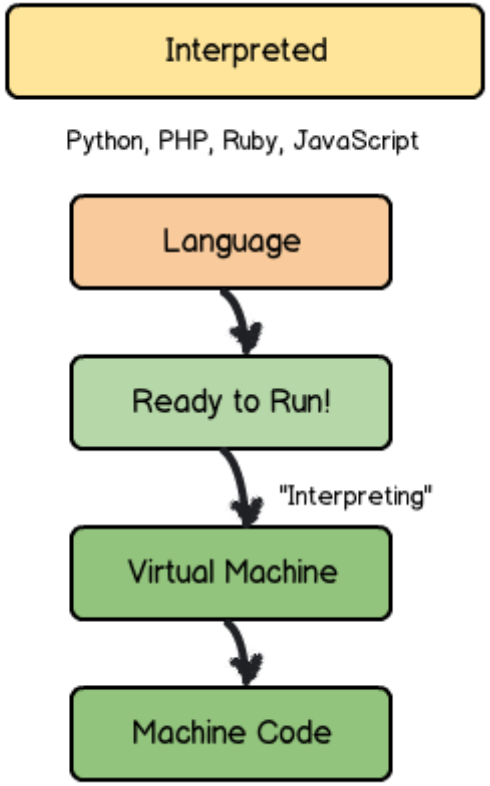
  - use computation friendly language (*e.g.*, Compiling to C)

Compiled Language

- Compiled approach gives a significant speedups







https://medium.com/@astermanuelg/blurred-lines-is-ruby-an-interpreted-language-2d3d6bca3d37
https://medium.com/@nicerajanikant/compiler-what-and-how-202cc661ae2b

Compiled Language

- Compiled approach gives a significant speedups (*usually*.)
- Possible Part
  - Loop that repeats the same operations many times.

    (Interpreter lang. probably use and create lots of temp. objects)
- Difficult Part
  - Regular expression and string operation
  - Call database
  - I/O related code
  - Vectorized `numpy` routine (It might be already highly optimized.)

JIT v.s. AOT

Two types of Compilers

- Ahead of Time (AOT) Complier
  - Cython
  - Create a static library that is specialized to our machine.

    (We have to build a code before using it.)
- Just In Time (JIT) Compiler
  - Numba, PyPy
  - Do not need compiling code before.
  - Compile a code <u>at the time of use</u>. “cold start”
  - Not good for <u>short, but good for frequently running scripts</u>.

Trade-off between AOT and JIT

- AOT is better, but this requires more human-efforts.
- JIT is relatively easy-to-use, but it has cold start problem.

```
memory usage: 86.1 KB

[3]: %%time
     fit = feols(fml="Y~X1 | f1", data=data, vcov="HC1")

     CPU times: user 2.69 s, sys: 196 ms, total: 2.89 s
     Wall time: 2.89 s

[4]: %%time
     fit2 = feols(fml="Y~X1 | f1", data=data, vcov="HC1")

     CPU times: user 7.58 ms, sys: 0 ns, total: 7.58 ms
     Wall time: 6.62 ms

[5]: %%time
     fit1 = feols(fml="Y~X1 + C(f1)", data=data, vcov="HC1")

     CPU times: user 15.2 ms, sys: 28.1 ms, total: 43.3 ms
     Wall time: 13.5 ms

[6]: %%time
     fit2 = feols(fml="Y~X1 + C(f1)", data=data, vcov="HC1")

     CPU times: user 32.5 ms, sys: 197 ms, total: 229 ms
     Wall time: 14.7 ms
```

https://github.com/s3alfisc/pyfixest/issues/284

C Compiler

- GNU ("GNU's Not Unix") C Compiler : gcc, g++
  - 재귀적 약자(recursive acronym) G = GNU, N = Not, U = Unix)
  - History: "GNU는 유닉스가 아니다."
    하지만 유닉스와 호환되는 자유 소프트웨어 운영체제를 만들겠다 라는 목표
    리처드 스톨만(Richard Stallman)이 1983년에 시작한 GNU 프로젝트.
  - GNU는 유닉스를 모방했지만 상표권·라이선스 제약 없이 누구나 쓸 수 있는 자유 소프트웨어를 지향

- Intel : icc

- Microsoft : cl

They (icc, cl) are often possible to squeeze out more performance by using a tuned compiler
(e.g., Intel's icc may produce faster code than gcc on Intel devices.)

- gcc is a good choice for most platforms; well supported and quite advanced.

Cython

>>> pip3 install Cython

Revisit Julia Set

```python
# area of complex space to investigate
x1, x2, y1, y2 = -1.8, 1.8, -1.8, 1.8
c_real, c_imag = -0.8, 0.156

def calculate_julia_set(maxiter, zs, cs):
  """Calculate output list using Julia update rule"""
  output = [0] * len(zs)
  for i in range(len(zs)):
    n = 0
    z = zs[i]
    c = cs[i]
    while abs(z) < 2 and n < maxiter:
      z = z * z + c
      n += 1
    output[i] = n
  return output

def build_julia_set(desired_width, max_iterations):
  """Create a list of complex coordinates (zs) and complex parameters (cs), build Julia set"""
  x_step = (x2 - x1) / desired_width
  y_step = (y1 - y2) / desired_width
  x = []
  y = []
  ycoord = y2
  while ycoord > y1:
    y.append(ycoord)
    ycoord += y_step
  xcoord = x1
  while xcoord < x2:
    x.append(xcoord)
    xcoord += x_step
  zs = []
  cs = []
  for ycoord in y:
    for xcoord in x:
      zs.append(complex(xcoord, ycoord))
      cs.append(complex(c_real, c_imag))

  output = calculate_julia_set(max_iterations, zs, cs)

if __name__ == "__main__":
_ build_julia_set(desired_width=2000, max_iterations=500)
```

```
>>> python3 -m timeit -n 5 -r 1 -s "import julia1_nocomment" "julia1_nocomment.build_julia_set (desir
ed_width=1000, max_iterations=300)”
>>> 5 loops, best of 1: 3.89 sec per loop
```

Cython (http://cython.org/)

- Cython is a compiler that converts type-annotated Python into a compiled extension module.
  - The type annotations are C-like.
  - Getting started is simple, but a learning curve is getting steep with each additional level of complexity and optimization. (시작은 쉽지만 응용 난이도는 어렵다.)
  - Good choice for calculation-bound functions
    1) Wide usage
    2) Maturity (lots of user and we can find many solutions in Google.)
    3) OpenMP support

*Libraries that use Cython: SciPy, scikit-learn, lxml, ZeroMQ

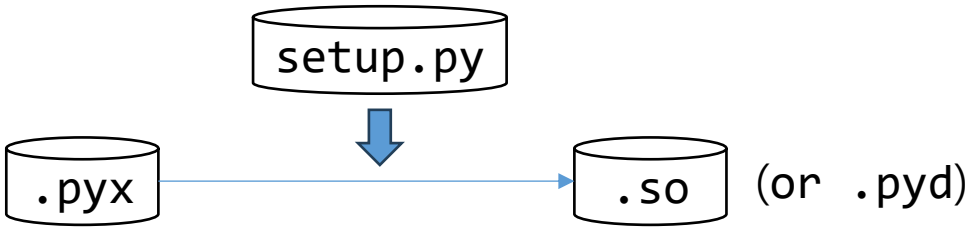
Compiling Python code using Cython

- There are three files that involves building a compiled extension module.

  1) The function to be compiled in a new `.pyx` file.

  2) `setup.py` script which convert `*.pyx` file into C object file.

     (on Unix-like system `.so` file; on Windows `.pyd` (DLL-like python lib.))



  3) Calling python code (e.g., building_julia_set)

Direct Compiling, Step1: write ‘cythonfn.pyx’

- Unmodified pure python code (only extension is changed.)

```
def calculate_julia_set(maxiter, zs, cs):
  output = [0] * len(zs)
  for i in range(len(zs)):
    n=0
    z = zs[i]
    c = cs[i]
    while n < maxiter and abs(z) < 2:
      z = z * z + c
      n+=1
    output[i] = n
  return output
```

Direct Compiling, Step 2: write 'setup.py'

- Compiling

```
from distutils.core import setup
from Cython.Build import cythonize

setup(ext_modules=cythonize("cythonfn.pyx",
                            compiler_directives={"language_level": "3"}))
```

from setuptools import setup

https://docs.python.org/ko/3.8/distutils/apiref.html

더 보기:

setuptools의 신규 및 변경된 setup.py 인자

`setuptools` 프로젝트는 `setup` 함수와 기타 API에 새로운 기능을 추가하고, API를 여러 파이썬 버전에서 일관되게 만들어서, `distutils`를 직접 사용하는 것보다 권장됩니다.

참고: 이 문서는 https://setuptools.readthedocs.io/en/latest/setuptools.html 의 `setuptools` 설명서가 현재 여기에 포함된 모든 관련 정보를 독립적으로 다루기 전까지만 보존됩니다.

```
>>> python3 setup.py build_ext --inplace
```

cythonfn.cpython-310-aarch64-linux-gnu.so

cythonfn.c

build\temp.linux-aarch64-3.10cythonfn.o

Detail information for options:
https://cython.readthedocs.io/en/latest/src/userguide/source_files_and_compilation.html#in-setup-py

Direct Compiling. Step 3: import the object (.so) file

```python
import time
import cythonfn
x1, x2, y1, y2 = -1.8, 1.8, -1.8, 1.8
c_real, c_imag = -0.8, 0.156

def build_julia_set(desired_width, max_iterations):
  x_step = (x2 - x1) / desired_width
  y_step = (y1 - y2) / desired_width
  x = []
  y = []
  ycoord = y2
  while ycoord > y1:
    y.append(ycoord)
    ycoord += y_step
  xcoord = x1
  while xcoord < x2:
    x.append(xcoord)
    xcoord += x_step
  zs = []
  cs = []
  for ycoord in y:
    for xcoord in x:
      zs.append(complex(xcoord, ycoord))
      cs.append(complex(c_real, c_imag))

  output = cythonfn.calculate_julia_set(max_iterations, zs, cs)

if __name__ == "__main__":
  build_julia_set(desired_width=2000, max_iterations=500)
```

Direct Compiling, Time Comparison

```
pure_python# python3 julia1_nocomment.py
>>> 3.585569381713867  sec

cython# python3 julia1_nocomment.py
>>> 1.613407850265503  sec
```

*Apple silicon has still some problem.
*AMD ryzen has still some problem.

- We improve the code performance without very little effort.

pyximport

Indirect Compiling, pyximport

```
import time
import pyximport
pyximport.install(language_level=3)
import cythonfn
x1, x2, y1, y2 = -1.8, 1.8, -1.8, 1.8
c_real, c_imag = -0.8, 0.156

def build_julia_set(desired_width, max_iterations):
  x_step = (x2 - x1) / desired_width
  y_step = (y1 - y2) / desired_width
  x = []
  y = []
  ycoord = y2
  while ycoord > y1:
    y.append(ycoord)
    ycoord += y_step
  xcoord = x1
  while xcoord < x2:
    x.append(xcoord)
    xcoord += x_step
  zs = []
  cs = []
  for ycoord in y:
    for xcoord in x:
      zs.append(complex(xcoord, ycoord))
      cs.append(complex(c_real, c_imag))
  start_time = time.time()
  output = cythonfn.calculate_julia_set(max_iterations, zs, cs)
  print(time.time()-start_time, " sec")

if __name__ == "__main__":
  build_julia_set(desired_width=1000, max_iterations=300)
```

- Do not need to write `setup.py`, but result is almost same.

  >>> 1.600527286529541 sec

Adding Type Annotations

Cython annotation : 1.type_casting (1)

- Cython has an annotation option that generate HTML file to show *"which code lines take a lot of time."*

>>> cython -a cythonfn.pyx

- 짙은 색은 파이썬 가상머신에서 더 많은 호출되었다는 것을 표시한다. 즉, 옅은 색은 C 코드가 더 많다는 의미.
- 2번째 줄은 반복계산 부분이 아니라 바꾸면 좋지만, 함수 자체의 입력, 출력을 고쳐야함.
- 함수 내에서 7,8,9 번째 줄이 가장 많이 호출되고 있으며, 그 다음으로는 3,10 번째 줄이 많이 호출되고 있음.
- C 로 컴파일 했음에도 불구하고 여전히 가상머신 호출이 많음 (**변수 타입 검사**)

```
Generated by Cython 0.29.32

Yellow lines hint at Python interaction.
Click on a line that starts with a "+" to see the C code that Cython generated for it.

Raw output: cythonfn.c

+01: def calculate_julia_set(maxiter, zs, cs):
+02:   output = [0] * len(zs)
+03:   for i in range(len(zs)):
+04:     n=0
+05:     z = zs[i]
+06:     c = cs[i]
+07:     while n < maxiter and abs(z) < 2:
+08:       z=z*z+c
+09:       n+=1
+10:     output[i] = n
+11:   return output
```

cythonfn.html

```
root@docker   -desktop:/workspace/hpp/w3# kernprof              -l -v julia1.py
Wrote profile results to julia1.py.lprof
Timer unit: 1e       -06 s


Total time: 34.4074 s
File: julia1.py
Function:      calculate_julia_set           at line 20


Line #            Hits              Time    Per Hit        % Time    Line Contents
==============================================================================
     20                                                             @profile
     21                                                             def calculate_julia_set(maxiter, zs, cs):
     22                                                                 """Calculate output list using Julia update rule"""
     23                1          3954.0     3954.0          0.0       output = [0] * len(zs)
     24          1000001        288293.0        0.3          0.8       for i      in range(len(zs)):
     25          1000000        280080.0        0.3          0.8           n = 0
     26          1000000        319092.0        0.3          0.9           z = zs[i]
     27          1000000        289383.0        0.3          0.8           c = cs[i]
     28         34219980      12785364.0        0.4         37.2           while abs(z) < 2 and n < maxiter:
     29         33219980      10350186.0        0.3         30.1               z = z * z + c
     30         33219980       9772526.0        0.3         28.4               n += 1
     31          1000000        318490.0        0.3          0.9           output[i] = n
     32                1           4.0          4.0          0.0       return output
```

Cython annotation : 1.type_casting (2)

- `cdef` : declare variable type inside a function.

```
def calculate_julia_set(maxiter, zs, cs):
  cdef unsigned int i, n
  cdef double complex z, c
  output = [0] * len(zs)
  for i in range(len(zs)):
    n=0
    z = zs[i]
    c = cs[i]
    while n < maxiter and abs(z) < 2:
      z=z*z+c
      n+=1
    output[i] = n
  return output
```

Cython annotation : 1.type_casting (3)

>>> cython -a cythonfn.pyx

타입 케스팅만 변경했음에도 성능 향상!

Generated by Cython 0.29.32

Yellow lines hint at Python interaction.
Click on a line that starts with a "+" to see the C code that Cython generated for it.

Raw output: cythonfn.c

```
+01: def calculate_julia_set(maxiter, zs, cs):
 02:   cdef unsigned int i, n
 03:   cdef double complex z, c
+04:   output = [0] * len(zs)
+05:   for i in range(len(zs)):
+06:     n=0
+07:     z = zs[i]
+08:     c = cs[i]
+09:     while n < maxiter and abs(z) < 2:
+10:       z=z*z+c
+11:       n+=1
+12:     output[i] = n
+13:   return output
```

cythonfn.html

>>> 0.4611778259277344 sec

Cython annotation : 2.equation

- Simplifying the equation for reducing python VM dependency.

```
def calculate_julia_set(maxiter, zs, cs):
  cdef unsigned int i, n
  cdef double complex z, c
  output = [0] * len(zs)
  for i in range(len(zs)):
    n=0
    z = zs[i]
    c = cs[i]
    while n < maxiter and (z.real*z.real+z.imag*z.imag) < 4:
      z=z*z+c
      n+=1
    output[i] = n
  return output
```

cythonfn.html

>>> 0.3110673427581787 sec

Cython with Numpy

Cython with Numpy: memoryview (1)

- This allows the same low-level access to any object that implements the buffer interface.

Cython이 numpy 배열에 접근하기 위해 memoryview 방식을 활용하면 손쉽게 저장된 데이터에 접근할 수 있다.

```
import numpy as np
cimport numpy as np

def calculate_julia_set(int maxiter, double complex[:] zs, double complex[:] cs):
  cdef unsigned int i, n
  cdef double complex z, c
  cdef int[:] output = np.empty(len(zs), dtype=np.int32)
  for i in range(len(zs)):
    n = 0
    z = zs[i]
    c = cs[i]
    while n < maxiter and (z.real*z.real+z.imag*z.imag) < 4:
      z = z * z + c
      n += 1
    output[i] = n
  return output
```

cythonfn.pyx

Cython with Numpy: memoryview (2)

- Update the calling function

```
typedef struct {
  struct __pyx_memoryview_obj *memview;
  char *data;
  Py_ssize_t shape[8];
  Py_ssize_t strides[8];
  Py_ssize_t suboffsets[8];
} __Pyx_memviewslice;
```

__Pyx_memviewslice object

To access the memory

(*((double *) (/* dim=0 */ (__pyx_v_a.data + __pyx_t_2 * __pyx_v_a.strides[0]))));

>>> 0.08209490776062012 sec

- The memoryview contains a C pointer some some data.
- It (shape, strides and suboffsets) allows Cython to do indexing the using the r aw data pointers and some simple C maths (which is very efficient.)

* Note that data converting time is not considered.

```
import time
import cythonfn
import numpy as np

x1, x2, y1, y2 = -1.8, 1.8, -1.8, 1.8
c_real, c_imag = -0.8, 0.156

def build_julia_set(desired_width, max_iterations):
  x_step = (x2 - x1) / desired_width
  y_step = (y1 - y2) / desired_width
  x = []
  y = []
  ycoord = y2
  while ycoord > y1:
    y.append(ycoord)
    ycoord += y_step
  xcoord = x1
  while xcoord < x2:
    x.append(xcoord)
    xcoord += x_step
  zs = []
  cs = []
  for ycoord in y:
    for xcoord in x:
      zs.append(complex(xcoord, ycoord))
      cs.append(complex(c_real, c_imag))
  zs = np.asarray(zs)
  cs = np.asarray(cs)
  start_time = time.time()
  output = cythonfn.calculate_julia_set(max_iterations, zs, cs)
  print(time.time()-start_time, " sec")

if __name__ == "__main__":
  build_julia_set(desired_width=1000, max_iterations=300)
```

Cython with OpenMP

Easy Parallelizing Solution with OpenMP (1)

- Open Multi-Processing (OpenMP) is a well-defined cross-platform API for C, C++, and Fortran.
  - Generate parallelized low-level code at complier level without relatively little effort.
  - Add Keyword in python code: `prange` (parallel range)
  - Add compiler directive in `setup.py`: `-fopenmp`
- `prange` disable GIL. (Global Interpreter Lock)
- Because GIL protects access to Python objects, disable GIL might lead to corrupt memory, when multiple-threads accesses simultaneously the memory

Easy Parallelizing Solution with OpenMP (2)

.pyx file

```
from cython.parallel import prange
import numpy as np
cimport numpy as np

def calculate_julia_set(int maxiter, double complex[:] zs, double complex[:] cs):
  cdef unsigned int i, length
  cdef double complex z, c
  cdef int[:] output = np.empty(len(zs), dtype=np.int32)
  length = len(zs)
  with nogil:
    for i in prange(length, schedule="guided"):
      z = zs[i]
      c = cs[i]
      output[i] = 0
      while output[i] < maxiter and (z.real*z.real+z.imag*z.imag) < 4:
        z = z * z + c
        output[i] += 1
  return output
```

Race condition
Schedule="static" , 부하를 균등하게 쪼갬, 쓰레드 대기 시간 발생
Dynamic / guided 가변적

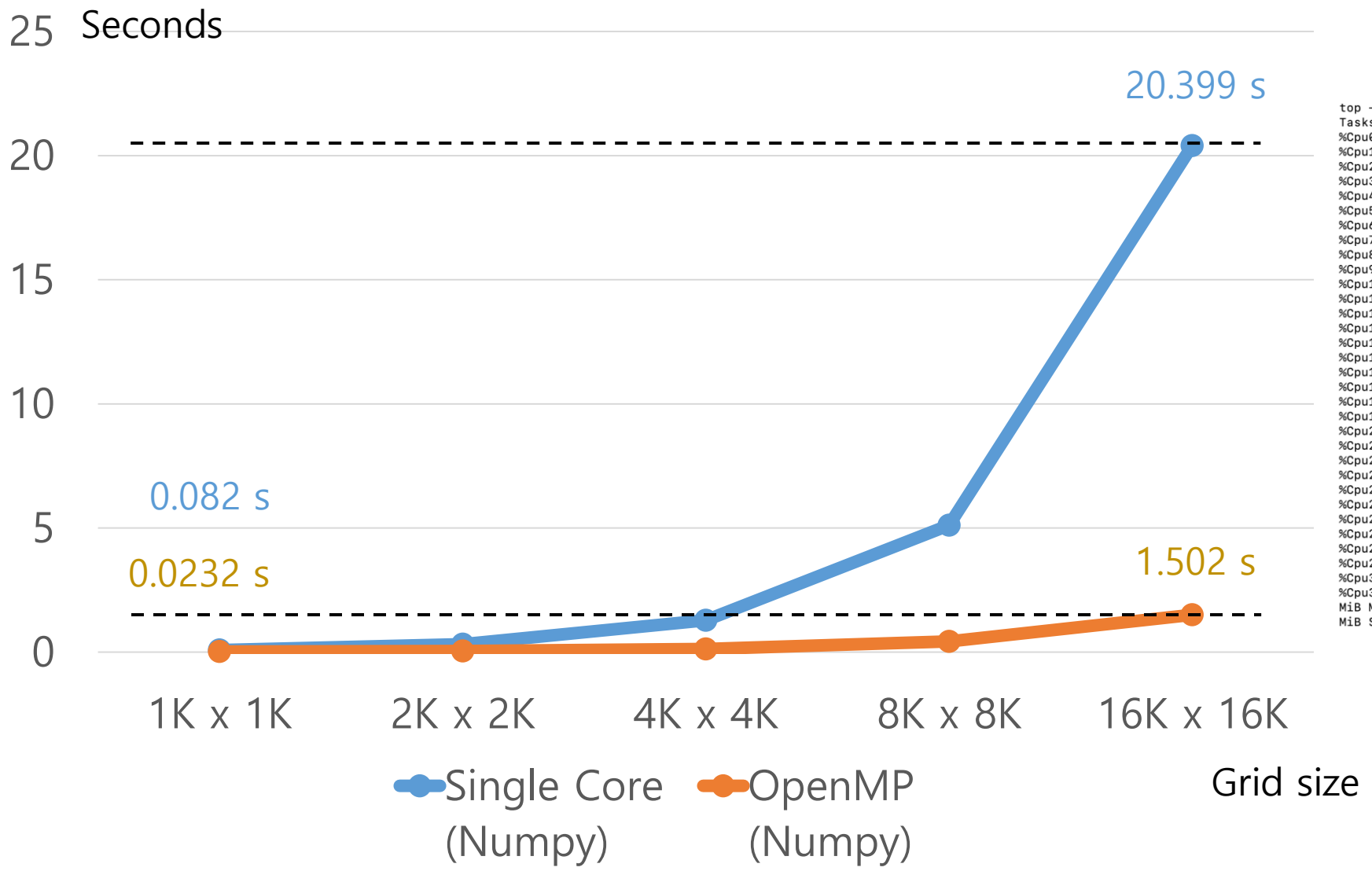
>>> 0.023244857788085938 sec

Easy Parallelizing Solution with OpenMP (3)

setup file

```
from setuptools import setup, Extension
from Cython.Build import cythonize
import numpy as np

ext_modules = [Extension("cythonfn",
                         ["cythonfn.pyx"],
                         extra_compile_args=['-fopenmp'],
                         extra_link_args=['-fopenmp'])]

setup(ext_modules=cythonize(ext_modules,
                            compiler_directives={"language_level": "3"}),
                            include_dirs=[np.get_include()])
```

Performance Comparison



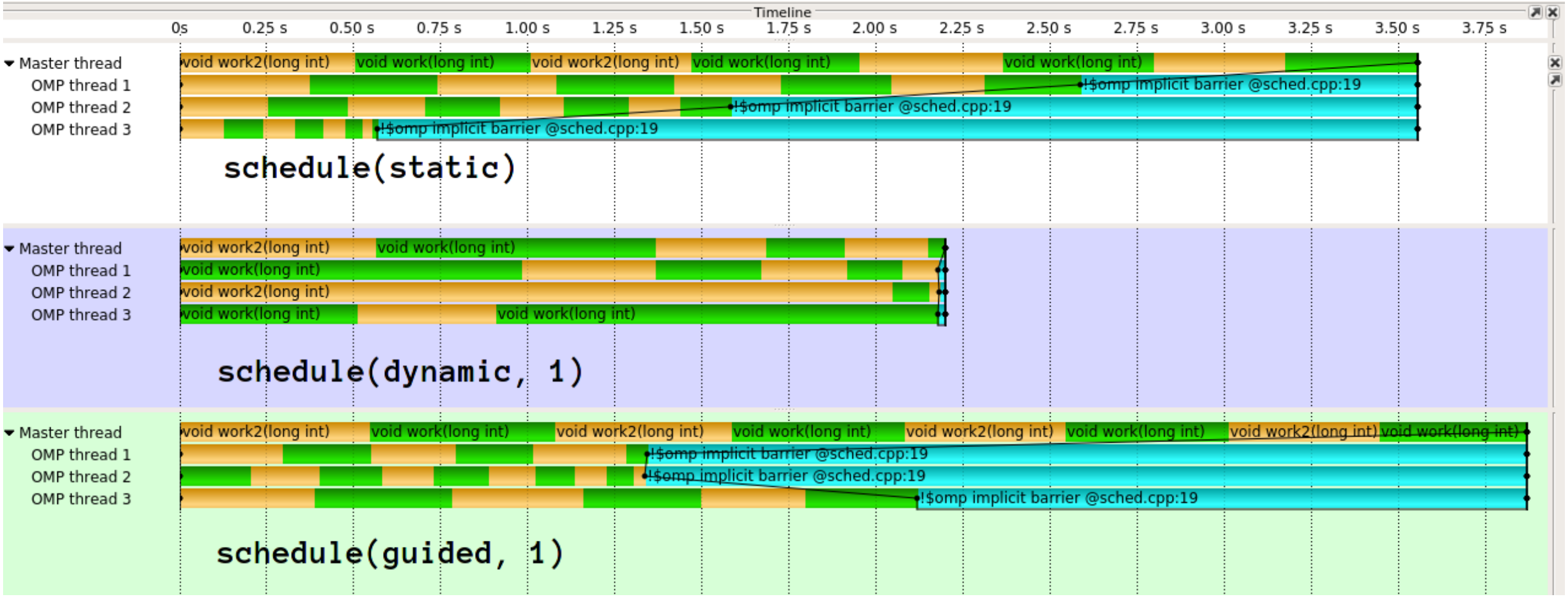

```
top - 10:38:26 up 44 days, 16:23,  5 users,  load average: 0.81, 0.63, 0.58
Tasks: 704 total,   2 running, 662 sleeping,  40 stopped,   0 zombie
%Cpu0  :  0.0 us,  0.0 sy,  0.0 ni, 96.2 id,  0.0 wa,  0.0 hi,  3.8 si,  0.0 st
%Cpu1  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu2  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu3  :  0.0 us,  0.0 sy,  0.0 ni, 99.7 id,  0.3 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu4  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu5  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu6  :  0.3 us, 12.5 sy,  0.0 ni, 87.1 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu7  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu8  :  0.0 us,  0.3 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu9  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu10 : 92.7 us,  7.3 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu11 :  0.3 us,  0.0 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu12 :  0.3 us,  0.3 sy,  0.0 ni, 99.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu13 :  0.7 us,  0.0 sy,  0.0 ni, 99.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu14 :  0.3 us,  0.3 sy,  0.0 ni, 99.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu15 :  0.3 us,  0.3 sy,  0.0 ni, 99.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu16 :  0.3 us,  0.3 sy,  0.0 ni, 99.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu17 :  0.0 us,  0.3 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu18 :  0.0 us,  0.3 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu19 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu20 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu21 :  0.3 us,  0.3 sy,  0.0 ni, 99.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu22 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu23 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu24 :  0.0 us,  0.3 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu25 :  0.3 us,  0.7 sy,  0.0 ni, 99.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu26 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu27 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu28 :  0.3 us,  0.0 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu29 :  0.0 us,  0.7 sy,  0.0 ni, 99.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu30 :  0.0 us,  0.7 sy,  0.0 ni, 99.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu31 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 128473.2 total,  27592.6 free,  33025.6 used,  67855.0 buff/cache
MiB Swap:   2048.0 total,   2008.7 free,     39.2 used.  93627.1 avail Mem
```

Load Balancing: Schedule

static	dynamic	guided
The workload is distributed evenly across the available CPUs.	Allocating workload in smaller **chunks** dynamically at runtime. *chunk: iteration numbers for each thread.	
Some regions will complete faster than others, and those threads will then sit idle.	The chuck size is set to small (default 1,) then the workload is allocated to each thread. An idle thread takes a work sequentially until disappearing all tasks. Lots of context switching!	At first, the chuck size is set to big, then the chuck size is gradually decreased according to the workload is smaller. Relatively small context switching and dynamic allocation.

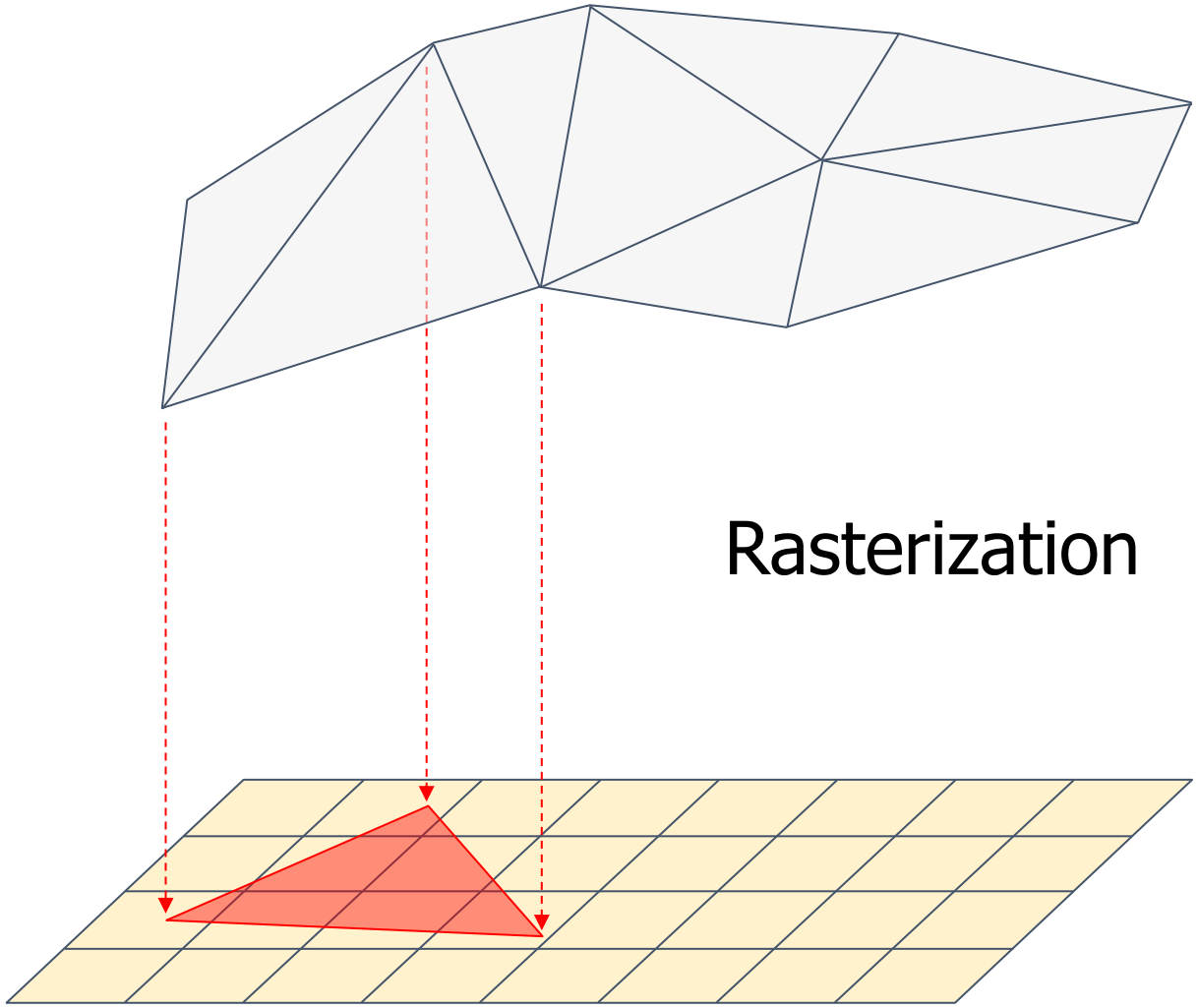
Load Balancing: Schedule

- Do not guess, measure it!



https://stackoverflow.com/questions/42970700/openmp-dynamic-vs-guided-scheduling

Another example : rasterization



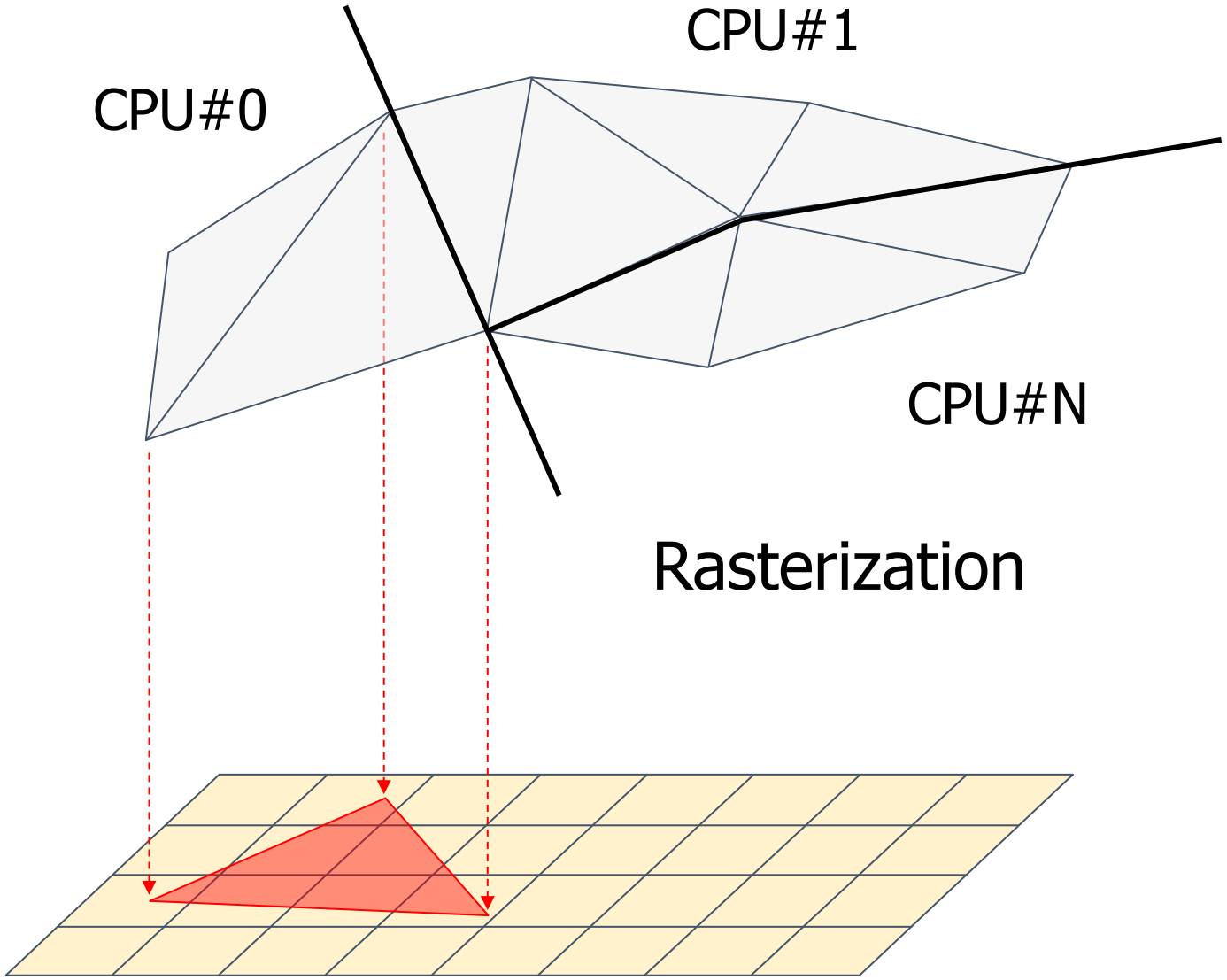
3D mesh information
- Continuous data
- # meshes : ~10K
- # points : ~5K

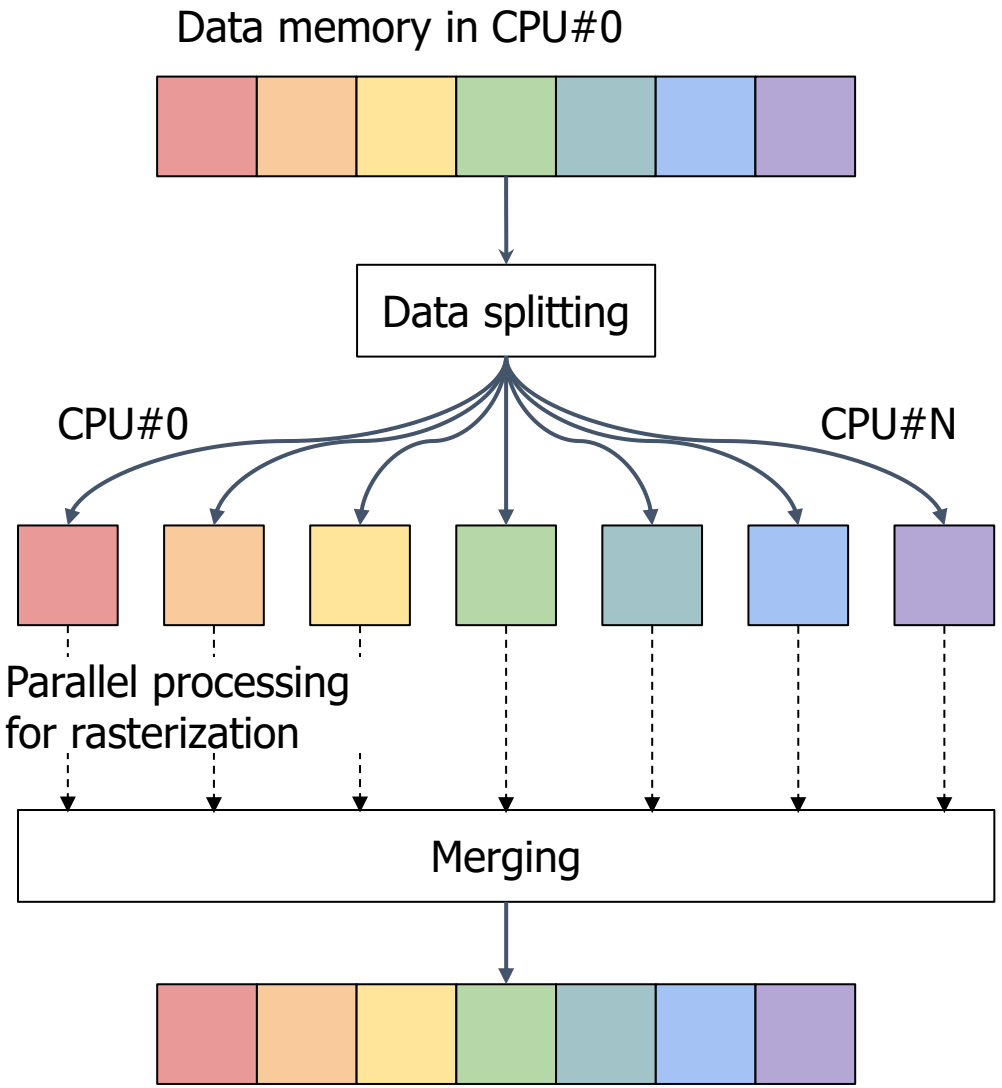
2D grid-map
- Discretized data (resolution: 20cm)
- # grids : (21m x 40m)

local_costmap2d
- Discretized data (resolution: 5cm)
- # grids : (40m x 40m)

Rasterization for inner triangles and edges of all meshes!
- Computation time w/o parallel processing

Another example : rasterization





Numba

```
>>> pip3 install numba
```

JIT Compiler

- Numba is a just-in-time complier (compile at runtime) which specialized `numpy` code.
  - Quick results.
  - Automatically generate GPU codes. (Cython cannot)
  - But, Numba does not bind to external C libraries (Cython do)
  - But, Numba has many dependencies.

Numba

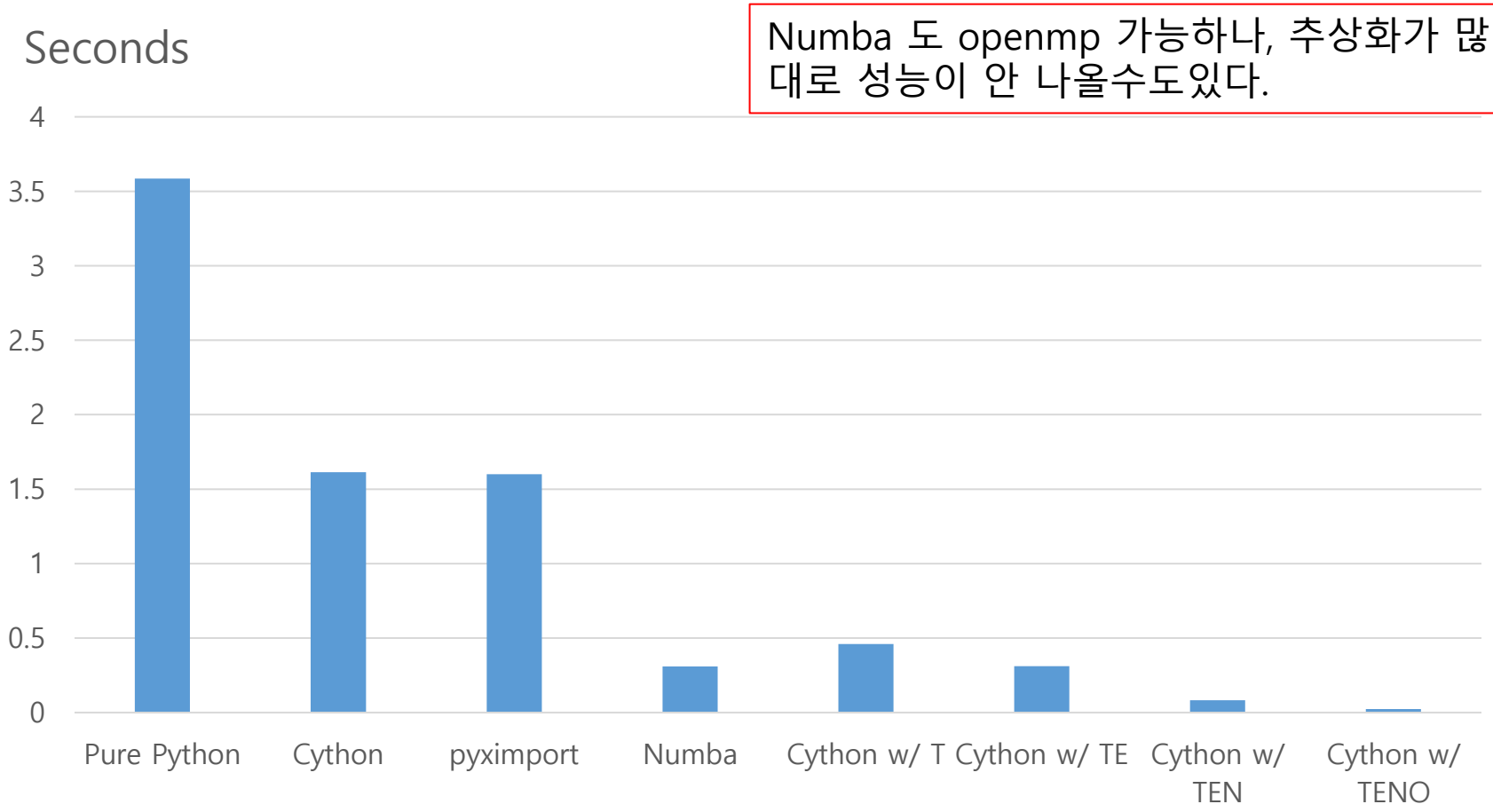
```python
import time
import numpy as np
from numba import jit

x1, x2, y1, y2 = -1.8, 1.8, -1.8, 1.8
c_real, c_imag = -0.8, 0.156

@jit()
def calculate_julia_set(maxiter, zs, cs, output):
  for i in range(len(zs)):
    n = 0
    z = zs[i]
    c = cs[i]
    while n < maxiter and (z.real*z.real + z.imag*z.imag) < 4:
      z = z * z + c
      n += 1
    output[i] = n
  return output
```

```python
def build_julia_set(desired_width, max_iterations):
  x_step = (x2 - x1) / desired_width
  y_step = (y1 - y2) / desired_width
  x = []
  y = []
  ycoord = y2
  while ycoord > y1:
    y.append(ycoord)
    ycoord += y_step
  xcoord = x1
  while xcoord < x2:
    x.append(xcoord)
    xcoord += x_step
  zs = []
  cs = []
  for ycoord in y:
    for xcoord in x:
      zs.append(complex(xcoord, ycoord))
      cs.append(complex(c_real, c_imag))
  output = [0] * len(zs)
  output = np.asarray(output)
  zs = np.asarray(zs)
  cs = np.asarray(cs)
  start_time = time.time()
  output = calculate_julia_set(max_iterations, zs, cs, output)
  print(time.time() - start_time, " sec")
```

>>> 0.3087596893310547 sec

Summary

Numba 도 openmp 가능하나, 추상화가 많이 되어 있어서 생각대로 성능이 안 나올수도있다.




```
  T: type_casting
 TE: type_casting, simple equation
TEN: type_casting, simple equation, Numpy
TENO: type_casting, simple equation, Numpy, OpenMP
```

Conclusion

- Numba gives quick results, but it is highly abstracted so, might not work well on your code.
- Cython offers the best results and can be widely applied, but it requires more effort and an additional maintenance cost.
- Additional projects
  - PyPy. Good parallel processing solution for web project.
  - Pythran. SIMD, OpenMP support
  - PyCUDA. CUDA and OpenCL bind (It gives GPU access interface.)
  - pyTorch, Tensorflow